

Design and Implementation of Viterbi Encoding and Decoding Algorithm on FPGA

Muhammad Irfan, Syed Muslim Shah, Ahmad Khalil Khan, *University of Engineering and Technology Taxila, Pakistan.*

Abstract—In this paper we have presented the hardware design implementation of Viterbi encoding and decoding algorithm. We have developed our own approach for the design of a 2/3 bit rate encoder and decoder. Convolution coders which are designed with shift registers and modulo-2 adders have been outlined. Viterbi decoder section includes Branch Metric Unit (BMU), Add Compare Select Unit (ACSU), Normalization Unit, Decision Unit and Output Unit have been implemented for 2/3 bit rate. The design has been implemented on Virtex II FPGA using XILINX 6.1i software with supporting simulation tool Modelsim 5.7. The results obtained from synthesis, simulation and hardware testing were accurate, error-free and original information was recovered successfully.

Index Terms—Hamming distance, Maximum likelihood decoding, State diagram, Tree diagram, Trellis diagram, Viterbi encoder and decoder.

I. INTRODUCTION

VITERBI encoding is widely used for satellite and other noisy communication channels. There are two important components of channel using Viterbi encoding, A Viterbi encoder includes extra information in the transmitted signal to reduce the probability of errors in the received signal that may be corrupted by noise [1]. In digital communication system encoding and decoding are an important part of the system. Digital signal is input to the encoder, which encodes the message. The encoded message is transmitted through the transmitter after modulation and multiplexing. The Receiver receives this message which after demultiplexing and demodulation is input to decoder. The decoder decodes this message and we get the original message [8]. We have used convolution codes for developing encoder section.

Convolution encoding is used for development of encoder section. Using modulo 2 adders the information is transmitted. Due to noise if some bits are changed then in the decoding process this error is corrected. Encoding can be represented by state diagram, tree diagram, and trellis diagram [2]. Maximum likelihood decoding is used in Viterbi decoding section, thus applying maximum likelihood to the convolutionally encoded bits is performed in the context of choosing the most likely sequence [2]. Hard decision using hamming distances is

applied in the decoding process for evaluation of data transmitted [9].

This paper is organized as follows: Section II presents the Encoder hardware design for the rate 2/3. In Section III decoding algorithm is explained. Section IV deals with the hardware design of decoder and Section V is related to the results and conclusion is drawn in Section VI.

II. ENCODER HARDWARE DESIGN

In our project we considered rate 2/3 convolutional Encoder. The constraint length for the encoder is $K=4$. The block diagram is shown in Figure 1.

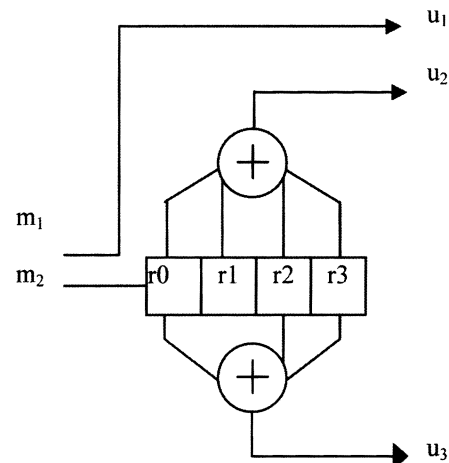


Fig. 1. Convolution Encoder for (Rate 2/3 K=4)

The encoder block encodes the data to be transmitted with defined bit rate. Encoder unit is designed with five edge triggered D-flip flops. Four flip flops were connected in series such that the output of one flip flop was fed to the other. The fifth flip flop was used to sequentially transmit the most significant bit of the input data. By using two exclusive or gates two outputs were obtained with every clock pulse. The third bit comes from the output of the fifth flip flop. This encoding information is also required at the receiver end while decoding the sequence, when traceback is performed on the trellis diagram. The logic diagram of the detailed encoder unit is shown in Figure 2.

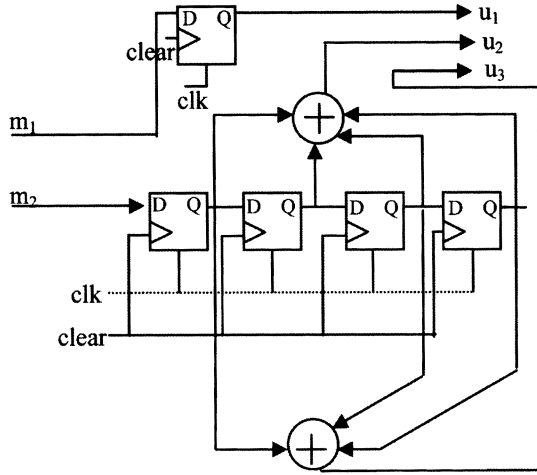


Fig. 2. Detailed Diagram of Encoder (Rate 2/3, K=4)

Modulo-2 operation means taking exclusive-or of the inputs to the Modulo-2 Adder.

$$u1 = m1 \quad (1)$$

$$u2 = r0 \oplus r1 \oplus r2 \oplus r3 \quad (2)$$

$$u3 = r0 \oplus r2 \oplus r3 \quad (3)$$

Convolution encoder can be represented in three different ways.

- State Diagram
- Tree Diagram
- Trellis Diagram

III. DECODING ALGORITHM

There are many different methods of decoding the received message at the receiver. This decoding must be very efficient and error-free otherwise the original message would be destroyed. The original message might get distorted on the way to receiver due to noise. It must be corrected by some means to get the correct message. Viterbi Algorithm performs maximum likelihood decoding. The algorithm involves calculating a measure of similarity, or distance, between the received signal, at time t_i and all the trellis paths entering each state at time t_i . These distances are known as Hamming distances. Hamming distance is equal to the number of bits, a bit combination differs from the other combination [2].

The Viterbi Algorithm removes from consideration those trellis paths that could not possibly be candidate for the maximum likelihood choice. When two paths enter the same state, the one having the least path metric is chosen. This path is called the surviving path. This selection of surviving paths is performed for all the states. The decoder continues in this way to advance deeper into the trellis, making decisions by eliminating the least likely paths. The early rejection of the unlikely paths reduces the decoding complexity [1] [2].

IV. DECODER HARDWARE DESIGN

A Viterbi decoder performs a maximum likelihood detection of data transmitted over a channel with inter-symbol interference (ISI). Decoder section consists of BMU, ACS, Decision unit, Normalization and Output unit.

A. Branch Metric Unit (BMU)

The branch metric unit calculates the hamming distance between the incoming data and the transition in the trellis. It is the first unit of the decoder section. The complete block diagram of the BMU is shown in the Figure 3. At each clock pulse the three data bits are received sequentially through the flip flops.

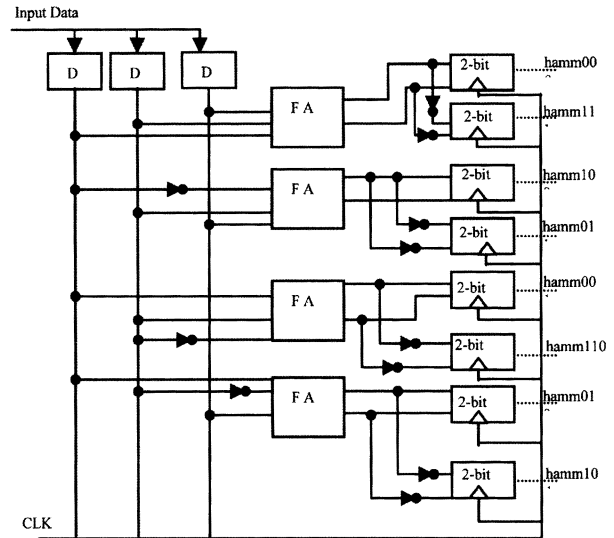


Fig. 3. Branch Metric Unit

Hamm 000: The difference between the incoming bits and the encoder output “000” is called the hamm000. Since the received bits are three so the maximum possible hamming distance is three. Two bits are required to represent the hamming distance. The truth table for the logic block for calculating the Hamm 000 is shown in Table I.

TABLE I
TRUTH TABLE FOR HAMM 000

u_1	u_2	u_3	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = u_1 \oplus u_2 \oplus u_3 \quad (4)$$

$$C = u_1 u_2 + u_2 u_3 + u_3 u_1 \quad (5)$$

The expressions for S and C are the same as for Boolean expressions for sum and carry of Full Adder (FA) so we can use FA for generating Hamm 000.

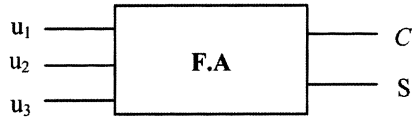


Fig. 4. Full Adder for Ham 000

Hamm 111: The hamming distance Hamm 111 will be obtained by complementing the hamming distance Hamm 000.

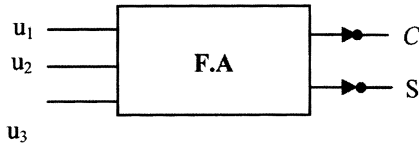


Fig. 5. Full Adder for Ham 111

Hamm 100: The difference between the incoming bits and the encoder output “100” is called the hamm100. The truth table for the logic block for calculating the hamm100 is shown in Table II.

TABLE II
TRUTH TABLE FOR HAMM 100

u_1	u_2	u_3	C	S
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

$$S = u_1' \oplus u_2 \oplus u_3 \quad (6)$$

$$C = u_1' u_2 + u_2 u_3 + u_3 u_1' \quad (7)$$

The expressions for S and C are the same as for Boolean expressions for sum and carry of FA with u_1 complemented so FA for generating Hamm100

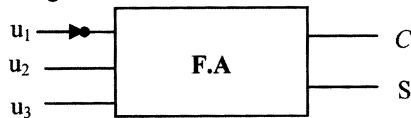


Fig. 6. Full Adder for Ham 100

Hamm 011: The hamming distance hamm011 will be obtained by complementing the hamming distance hamm100.

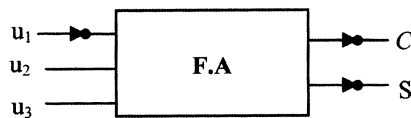


Fig. 7. Full Adder for Ham 011

Hamm 010: Similarly Hamm 010 can be calculated.

$$S = u_1 \oplus u_2' \oplus u_3 \quad (8)$$

$$C = u_1 u_2' + u_2' u_3 + u_3 u_1 \quad (9)$$

Hamm 101: It can be computed by complementing the output of Hamm 010.

Hamm 001: Similarly Hamm 001 can be calculated.

$$S = u_1 \oplus u_2 \oplus u_3' \quad (10)$$

$$C = u_1 u_2 + u_2 u_3' + u_3' u_1 \quad (11)$$

Hamm 110: It can be computed by complementing the output of Hamm 001

B. Add Compare Select Unit (ACSU)

This unit as the name indicates adds, compares and selects the path that is to be adopted. It is known from the butterfly architecture that there are always two paths by which a node could be reached [7]. First the sum of the branch metrics is calculated and the path metrics along these two paths using Full adders. Since there are four paths entering each node out of which we have to select one. If there are four numbers n_1, n_2, n_3, n_4 then for comparing them, six comparators are needed as shown in Figure 8.

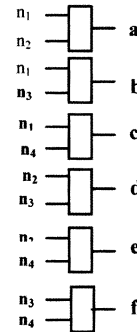


Fig. 8. Comparator for ACS Unit

TABLE III
TRUTH TABLE FOR ACS UNIT

a	b	c	d	e	f	S_1	S_0
0	0	0	x	x	x	0	0
1	x	x	0	0	x	0	1
X	1	x	1	x	0	1	0
X	x	1	x	1	1	1	1

$$S_1 = bdf' + cef \quad (12)$$

$$S_0 = ad'e' + cef \quad (13)$$

C. Normalization Unit

Going deeper into the trellis, the path metric keeps on increasing. Due to the repeated accumulation of branch metrics, the magnitude of these metrics is potentially unbounded [4]. Hence metric normalization schemes are

necessary for a fixed word length implementation. The full adders might not be able to accommodate these large values and overflow might occur. To avoid this situation a normalization unit is necessary. The Normalization unit is shown in Figure 9. When the most significant bit of all the eight registers becomes '1' the output of and gate becomes active high. The output of and gate is exclusive-or with the MSB of all the registers. Since the exclusive-or operation of any bit with '1' is always '0' this causes a '0' to be loaded into MSB of all the registers. This causes 8 to be subtracted from all the registers hence the mutual difference between all the registers have remained constant.

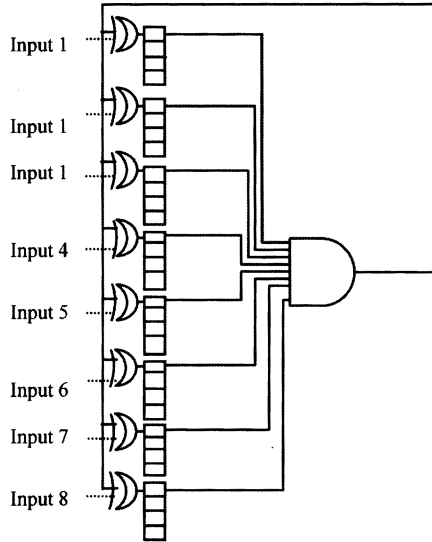


Fig. 9. Normalization Unit

TABLE IV
NORMALIZATION UNIT

Before Normalization		After Normalization	
8	1000	0	0000
9	1001	1	0001
10	1010	2	0010
11	1100	3	0011

D. Decision Unit

The Decision Unit makes the decision that from which node the traceback will be started. For this purpose it compares all the eight path metrics to decide which one is the smaller. The Comparison operation is performed in two steps. In the first step four out of eight path metrics are calculated by using four comparators and muxes. In the second step out of these four metrics the smallest one is calculated by using the six comparators. Using the information of first four and last six comparators the decision is made.

$$u = ab'c'g' + ade'h' + bdei + cefg \quad (14)$$

$$v = ad'e' + cef \quad (15)$$

$$w = bde' + cef \quad (16)$$

It was found that approximately after going 34 paths going deeper into trellis, Only one path for traceback is selected and all other path are discarded.

E. Output Unit

The output unit is the last and the most important unit of the Viterbi decoder. This unit is used to recover data bits using information from ACS unit and decision unit. This unit is Micro programmed control and the control program resides in the 64x5 ROM. The read address of the ROM is generated by a 6-bit counter. The ROM is used to generate the write addresses of the 32x16 RAM and the LIFO 32x2 RAM). The data which is written into RAM comes from two bit registers of ACS and is continuously written until the traceback is started. The traceback is started when we reach at a depth of 32 into the trellis. At that time the counter output is 35 the reason being that there is a latency of 3 clock pulses in the previous units. Similarly the next traceback will occur when the counter output will be 3. When the traceback is started the 3-bit register is loaded with the output of the Decision unit which shows the node having minimum path metric. However during other times it is continuously shifted towards left and a new bit is introduced into its rightmost position.

The output of the 3-bit register is used to select the required bits from the output of the RAM using two muxes. These bits serve two important purposes. First it decides what is the next state in the traceback process for example if the contents of the 3 bit register are shifted towards left and the output of the upper mux is accommodated in the rightmost position we get the next state. Secondly it helps in determining the data bits for example the output of the lower mix is the same as the data bit m1. The other data bit m2 is the same as the leftmost bit of the 3-bit register. The data bits recovered are stored into the LIFO the reason being that during traceback the 32nd input bits are recovered first and the 1st input bits are recovered at the last. When the LIFO is read the 1st input bits will first come out of the LIFO since they are written into the LIFO at the last.

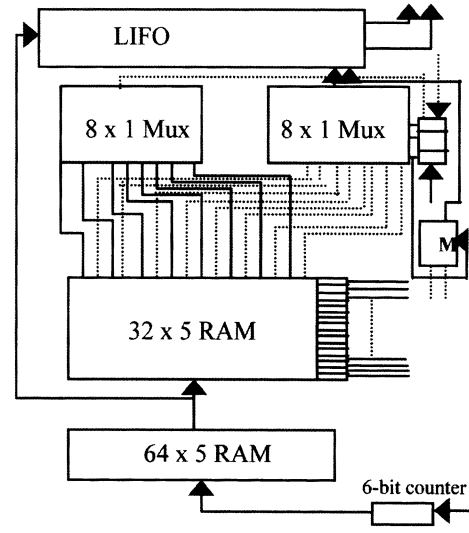


Fig. 10. Output Unit

V. RESULTS

A test bench program was written which instantiates a random generator to create a binary signal as an input to the convolutional encoder. The output of the encoder is provided to the Viterbi decoder block. The Viterbi decoder takes the distance measures and calculates the most likely transmitted signal. It does this by keeping a running history of the previously received signals in a path memory. By keeping a history of possible sequences and using the knowledge that the signals were generated by a state machine, it is possible to select the most likely sequences. Output of the test bench program is given in Table V. The optimum decoding method is maximum-likelihood decoding where the decoder attempts to find the closest valid sequence to the received bit stream. The possible received bit sequences form a trellis structure and the Viterbi Algorithm tracks likely paths through the trellis before choosing the most likely path. Xilinx Virtex II FPGA was used for hardware implementation.

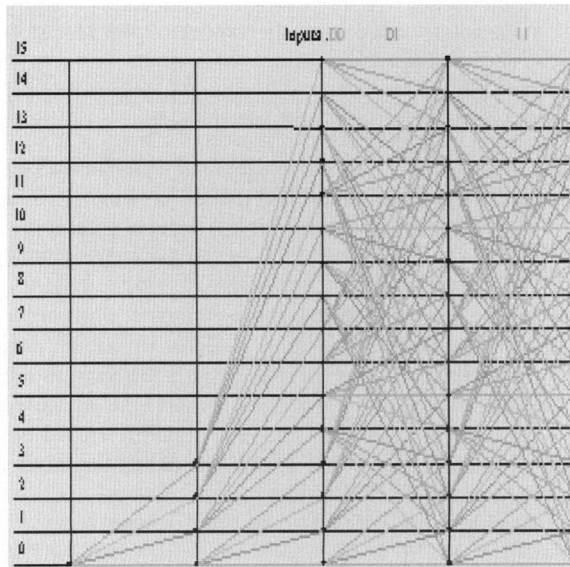


Fig. 11. Trellis diagram for Rate 2/3

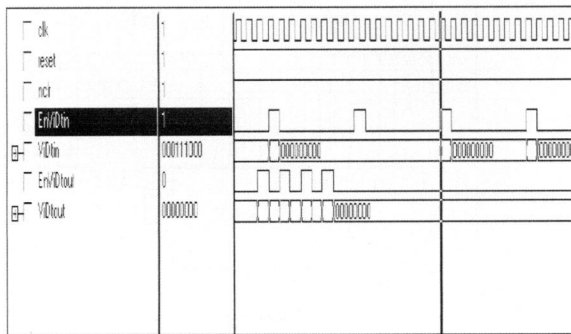


Fig. 12. Decoding The Sequence

TABLE V
OUTPUT FROM VITERBI TESTBENCH

t	clk	X	Y	Out	Error
0	0	3	x	x	0
50	1	3	x	x	0
51	1	0	x	x	0
60	1	0	0	0	0
100	0	0	0	0	0
150	1	0	0	0	0
151	1	1	2	0	0
1351	1	1	0	0	0
1400	0	1	0	0	0
1450	1	1	0	0	0
1451	1	2	5	2	0
1500	0	2	5	2	0
1550	1	2	5	2	0
1551	1	3	4	5	0

Figure 12 shows the Verilog Simulator output from the testbench.

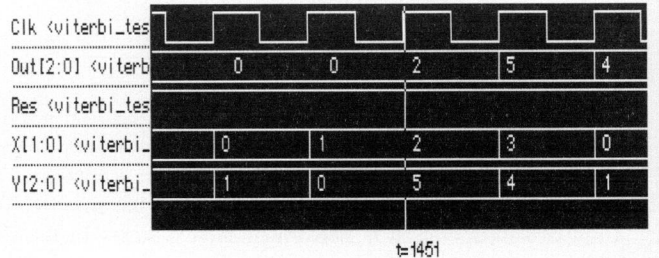
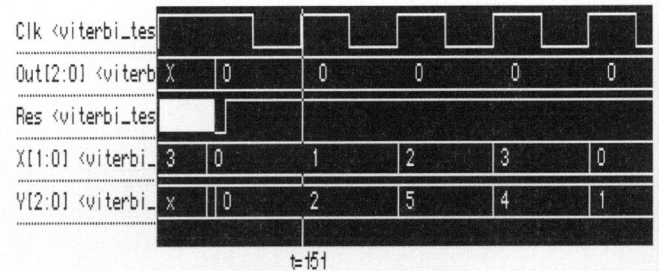


Fig. 13. Output Waveform

VI. CONCLUSION

Viterbi Algorithm allows safe data transmission via error correction and original message can be recovered accurately without any noise. It was concluded from research that if traceback is started after going deeper into trellis diagram then more accurate data can be achieved but it results in complex hardware design and latency in the received signal. Viterbi algorithm of any rate can be designed using same basic principles and techniques.

REFERENCES

- [1] Wesel, Rick. *Convolution Codes*, University of California Los Angeles, 2005
- [2] Bernard Sklar “*Digital Communications Fundamentals and applications*”, 2003, pp. 380-419.
- [3] S. Benedetto, D. Divsalar, G. Montorsi, and F. Pollara, A softinput soft-output APP module for the iterative decoding of concatenated codes, *IEEE Commun. Lett.* **1**(1): 22–24 (Jan.1997).
- [4] J. B. Anderson and K. Balachandran, Decision depths of convolutional codes, *IEEE Trans. Inform. Theory* **35**(2): 455–459 (March 1989).
- [5] C. Berrou, A. Glavieux, and P. Thitimajshima, Near Shannon limit error correcting coding and decoding: Turbo-codes, *Proc. Int. Conf. Communication*, May 1993, pp. 1064–1070.
- [6] L. R. Bahl, J. Cocke, F. Jelinek, and J. Raviv, Optimal decoding of linear codes for minimizing symbol error rate, *IEEE Trans. Inform. Theory* **20**(2): 248–287 (March 1974).
- [7] R. Johannesson and Z. Wan, A linear algebra approach to minimal convolutional encoders, *IEEE Trans. Inform. Theory* **39**(4): 1219–1233 (July 1993).
- [8] R. Johannesson and K. Sh. Zigangirov, *Fundamentals of Convolutional Coding*, IEEE Press, 1999.

ABOUT THE AUTHORS

Muhammad Irfan did his BS and MS in Electrical & Electronic Engineering from University of Engineering and Technology Taxila, Pakistan. He has more than three years of embedded hardware and software testing using HILS. His areas of interests are VLSI Design, Computer Architecture, Coding Theory and Microelectronics. He is employed as Electrical engineer in National Engineering and Scientific Commission, Islamabad, Pakistan.

Email: irfan29wah@yahoo.com

Syed Muslim Shah did his BS in Electrical & Electronics Engineering from NWFP, University of Engineering and Technology, Peshawar, Pakistan and student of MS Computer Engineering at UET, Taxila. He has more than tree years experience in design, development and testing of Embedded Systems Software, experience of real time software preparation and user interface applications development for testing embedded hardware and software. He is employed as engineer in National Engineering and Scientific Commission, Islamabad, Pakistan.

Email: syedmuslimshah@hotmail.com

Ahmad Khalil Khan did his B.Sc Electrical Engineering from University of Engineering and Technology Lahore, MS from Louisian State University (U.S.A). Currently he is enrolled for PH.D. His areas of interests are Digital Video Processing, Computer Architecture, Embedded Systems and Error Correcting Codes. He is currently working as a professor in Electrical Engineering Department, University of Engineering and Technology Taxila, Pakistan.

Email: profkhalil@uettaxila.edu.pk